

Leistungsbeurteilung 426

Modulnummer	426	Verordnung	2021	Anbieter Version	1.0
Titel	Software mit agilen Methoden entwerfen				
Autoren	Fabrice Thut <fabrice.thut@bbb Baden.ch> Manuel Bachofner <manuel.bachofner@bbb Baden.ch>				

Datum:

Teammitglied 1 (von Teammitglied 1 ausgefüllt)

Name:

Vorname:

Teammitglied 2 (von Teammitglied 2 ausgefüllt)

Name

Klasse:

Experte:

Note:

Hilfsmittel

Folgende Hilfsmittel sind erlaubt:
Alle Unterlagen

Folgende Hilfsmittel sind nicht erlaubt:
Kommunikationsmittel, wie Mobiltelefon, Messenger

Prüfungsdauer
Prüfungsinfrastruktur

~10 Lektionen zu 45 min
Folgende Infrastruktur steht zur Verfügung:
Eigenes Notebook
Gitlab (git.bbb Baden.ch)

Notenskala

Linear (Note = $1 + 5 \cdot \frac{\text{Erreichte Punktzahl}}{\text{Gesamte Punktzahl}}$), auf halbe Noten gerundet.
1 ist die schlechteste, 6 die beste Note.

Objekt

Funktionsfähiger, getesteter und dokumentierter Software-Release.

HZ	Handlungsziel	Aufgabe(n)
1	Vorgegebene Funktionalität im Rahmen eines Softwareprojekts mit einer agilen Methode umsetzen.	1
2	Funktionalitäten schrittweise mit Praktiken der agilen Entwicklung in vorgegebenen Release-Zyklen realisieren, testen und den Software-Release in Kurzform präsentieren.	3
3	Bestehende Entwurfsmuster und/oder geprüfte Softwarekomponenten zur Lösung von Problemen gezielt einsetzen.	2,3
4	Die Ergebnisse und das Arbeitsvorgehen eines Release-Zyklus reflektieren und daraus Erkenntnisse für das weitere Vorgehen ableiten.	5
5	Die Projektdokumente und den Programmquelltext in einem gemeinsamen Versionsverwaltungssystem bereitstellen	3
6	Programmquelltext nach Konventionen intuitiv verständlich formulieren und bei Bedarf überarbeiten.	3,4

Bewertung:

Aufgabe	Gewichtung	Punkte	Bewertung
Aufgabe 0-1:	1	3	Die administrativen Vorgaben sind eingehalten
Aufgabe 1-1:	2	3	Die User Story ist syntaktisch und semantisch korrekt und enthält eine eindeutige ID.
Aufgabe 1-2:	2	3	Es sind mindestens 6 User Stories vorhanden.
Aufgabe 2-1:	1	3	Die Begründung für die Wahl des Design Patterns ist korrekt.
Aufgabe 3-1:	2	3	Die Anforderungen an die Software werden erfüllt
Aufgabe 3-2:	2	3	Jedes Teammitglied arbeitet nachweislich an einem eigenen Branch.
Aufgabe 3-3:	1	3	Für jedes implementierte Feature existiert ein eigener, dokumentierter Branch. Jeder Branch enthält die ID der User Story. Branches werden in dem «main» zurück gemerged.
Aufgabe 3-4:	2	3	TDD wurde angewandt.
Aufgabe 3-5:	2	3	Das Observer-Pattern wurde korrekt implementiert.
Aufgabe 3-6:	2	3	Das vorgeschlagene Designpattern wurde eingesetzt.
Aufgabe 3-7:	1	3	Keine Code Smells vorhanden.
Aufgabe 4-1:	2	3	Die Ausführungen zu den SOLID Kriterien sind korrekt.
Aufgabe 5-1:	2	3	Die Dokumentation der Retrospektive ist zielgruppenorientiert.
Aufgabe 5-2:	1	3	Die Folgerungen aus der Retrospektive sind sinnvoll.

Punkte

Jede Teilaufgabe wird mit 3 Punkten bewertet. Die Totalpunktzahl ergibt sich aus der Multiplikation von Gewichtungsfaktor und erreichter Punktzahl pro Teilaufgabe.

Verspätete Abgaben

Abgaben welche nach dem Abgabetermin erfolgen, haben einen Notenabzug von 0.5 Note pro angebrochenen 24h zur Folge.

Hinweise:

- Diese Prüfung ist eine Partnerarbeit.
- Es dürfen keine alten Prüfungen und/oder Daten von alten Prüfungen verwendet werden.
- Betrügereien, wie Spicken o.Ä., oder Betrugsversuche führen zum Ausschluss aus der Prüfung.
- Im Falle eines Betrugs gilt die Leistungsbeurteilung als absolviert, aber nicht bestanden.
- Geben Sie diese Aufgabenstellung pro Gruppe und den Lösungsbogen mit Nachname, Vorname und Klasse beschriftet ab.
- Bezeichnen Sie Lösungen und Korrekturen klar und deutlich.
- Verwenden Sie einen dokumentenechten Stift (Tinte/Kugelschreiber).
- Wenn Sie Lösungen elektronisch oder auf Zusatzblättern abgeben, verweisen Sie auf diese Zusatzmaterialien auf Ihrem Aufgabenblatt.
- Alle Dateien müssen in einem Ordner abgelegt werden, dessen Name dem Format Name1Name2LB-426 entspricht. (z.B. MuellerMeierLB-426).
 - Die gepackte Datei muss gleich wie der Ordner benannt werden.
 - Programmierprojekte müssen gleich wie der Ordner benannt werden.
- Alle schriftlichen Ergebnisse geben Sie zusammengefasst in einem File (pdf oder docx) elektronisch ab.
- Folgende Formate sind bei den elektronischen Lösungen erlaubt:
 - Programmcode: Visual Studio-Projekte oder Dateien mit Ordnerhierarchie
 - Bilder: jpg, gif, png
 - Texte: ASCII-Texte, Microsoft Word, Adobe Acrobat pdf
 - Anderes: Microsoft Excel, Microsoft PowerPoint
- Die elektronischen Lösungen werden am Schluss der Prüfung auf Moodle hochgeladen.
- Elektronisch abgegebene Lösungen werden mit Vorteil gezippt.
- Sie sind verantwortlich dafür, dass Sie alle Dateien abgeben.
- Alle Daten dieser LB (Aufgabendateien und Lösungen) müssen sofort nach dem Abschluss der LB und dem Hochladen gelöscht werden.
- Vor Beginn der Prüfung abgegebene Daten:
 - Keine
- Als Programmiersprache muss C# verwendet werden. Ausnahmen von dieser Regel müssen mit der Lehrperson vereinbart werden.
- Regelung zu verspätet eingereichter Leistungsbeurteilungen:
 - Pro angefangene 24h Verspätung wird eine halbe Note (0.5 Notenpunkte) abgezogen. Verspätet eingereichte Leistungsbeurteilungen werden korrigiert, bewertet und dann mit dem Abzug der Verspätung verrechnet.
 - Aufschiebende Wirkungen haben nicht planbare Absenzen wie Krankheit oder Unfall, sofern Sie vom Berufslernenden bei Eintritt des Ereignisses kommuniziert wurden. Die Lehrperson entscheidet über die Abgabetermine.

I Ausgangslage

In dieser Leistungsbeurteilung arbeiten Sie in zu zweit an einem Software-Projekt. Dabei setzen Sie agile Praktiken ein und achten auf die Codequalität.

Es stehen verschiedene Projekte zur Auswahl. Die Zuordnung der Projekte und Gruppen ist in der Verantwortung der Lehrperson.

II Aufgaben

Halten Sie alle administrativen Vorgaben (unter anderem diejenigen auf Seite 3) ein.

1. User-Stories: Erstellen Sie mindestens sechs User Stories anhand der Anforderungen.
2. Designpatterns: Sie werden das Observer-Pattern implementieren müssen.
Notieren Sie ein weiteres Designpattern, welches Sie im Projekt verwenden werden.
Beschreiben Sie, wo Sie dieses verwenden wollen und begründen Sie den Einsatz.
3. Implementieren Sie das Projekt nach den folgenden Kriterien
 - a. Teilen Sie die Software auf und arbeiten Sie getrennt an eigenen Branches.
Dokumentieren Sie die Aufteilung im Lösungsdokument.
 - b. Verwenden Sie Git als Versionierungstool. Verwenden Sie die Gitlab-Installation der BBB für das Remote-Repository. Geben Sie das Projekt nur den Teammember und der Lehrperson frei (Maintainer Rolle).
 - c. Erstellen Sie für jedes Feature einen Branch. Der Name des Branches enthält die ID der User Story, die Sie damit realisieren. Dokumentieren Sie jeden Branch stichwortartig.
Nota Bene: Deaktivieren die Option «Enable "Delete source branch" option by default» im GitLab -> Projekt xy -> Settings -> Merge Requests.
 - d. Arbeiten Sie nach TDD. Committen und pushen Sie immer(!) Tests bevor Sie die Funktionalität implementieren.
 - e. Setzen Sie die Patterns ein, welche sie in Aufgabe 2 definiert haben.
 - f. Implementieren Sie professionell (Einhalten Coding Convention, SOLID, keine Code-Smells, etc.).
4. Beurteilen Sie die fertige Software nach den SOLID-Kriterien. Schlagen Sie Verbesserungen vor, falls Sie ein Kriterium verletzt haben sollten.
5. Führen Sie eine Retrospektive für das Projekt durch. Fokussieren Sie sich dabei nicht auf das Produkt (ihre Software), sondern auf ihre Praktiken und die Zusammenarbeit. Identifizieren Sie drei Hindernisse (Impediment) und schlagen Sie pro Hindernis eine für Sie realisierbare Lösung vor, welches das Hindernis aus dem Weg räumt. Dokumentieren Sie die Hindernisse und die Impediments in ihrem Lösungsdokument.

III Projekte

Casino



Die Projektteams erhalten den Auftrag, eine Casino-Software mit zwei Spielen. Der Nutzer muss nach starten der Anwendung einen frei gewählten Betrag in Jetons umwandeln, die dieser für den Einsatz in den Spielen benötigt. Der Spieler kann nach der Auswahl des Spiels dieses spielen, nach persönlichem Abschluss zur Auswahl zurückkehren und sich neu entscheiden oder aufhören. Die Einsätze erfolgen in Abhängigkeit der jeweiligen Spielregeln der Spiele.

Für dieses Projekt sollen zwei Spiele umgesetzt werden. Die Software soll so konzipiert sein, dass neue Spiele einfach integriert werden können.

Die Daten müssen nicht persistent gespeichert werden.

Folgende Auswahl an Spielen steht zur Verfügung:

1. Bingo
Simulation eines Bingo-Spiels gegen einen oder mehrere Computergegner. Wer zuerst «Bingo» hat, gewinnt das Spiel. Der Spieler kann den Einsatz und die Anzahl Spieler wählen. Die Gewinne erhöhen sich, je mehr Gegner mitmachen.
2. BlackJack
Blackjack-Spiel gegen einen oder mehrere Computergegner. Der Spieler kann den Einsatz und die Anzahl Spieler wählen. Die Gewinne erhöhen sich, je mehr Gegner mitmachen.
3. Roulette
Setzen von Beträgen auf mehrere Felder gleichzeitig. Es kann auf Farben oder Nummern gesetzt werden. Je nach Wahrscheinlichkeit ändern sich die Gewinnbeträge.
4. Slotmaschine
Die Slotmaschine besteht aus höher und niederwertigen Symbolen. Ein Spiel kostet einen fixen Betrag. Bei drei gleichen Symbolen werden je nach Symbol andere Beträge ausgezahlt.
5. Craps: Ein Spieler würfelt und gewinnt oder verliert den Eingesetzten Betrag:
Wirft der Shooter im ersten Wurf
 - die Augensumme 7 oder 11, so hat er ein Natural und gewinnt sofort.
 - die Augensumme 2, 3 oder 12, so ist dies ein Crap und er verliert sofort.
 - die Augensumme 4, 5, 6, 8, 9 oder 10, so ist die geworfene Augensumme sein Point, und der Shooter würfelt ein weiteres Mal.Ab dem zweiten Wurf gilt:
 - Wirft der Shooter dieselbe Augensumme wie im ersten Wurf, also seinen Point, so gewinnt er.
 - Wirft der Shooter die Augensumme 7, so verliert er.
 - Wirft er irgendeine andere Augensumme, so würfelt er ein weiteres Mal.
6. Sie haben eine eigene Idee? Sprechen Sie sich mit ihrer Lehrperson ab.

Textbasiertes Spiel

```
\ [Bori] [HP] 10/10 [Weapon] Dagger [Skill] Double Trouble /
[INVENTORY]
Food : Apple | Vanilla Cake | Apple | Bread |
Armor :
Weapon: Dagger |

+-----+
|Town Square|
+-----+
# You are in the middle of the square, with streets surrounding you from all
directions. There is many people around, they all seem busy

> A white fountain is streaming water
> An apple lies in the dirt
> A loaf of bread lies on ground

SOUTH| Butchery
EAST | Bakery
WEST | House 63 (Ground)

# Would you like to: <go>? <pick>? <look>? <eat>?
>
```

Sie erstellen ein textbasiertes Spiel. Darin existiert eine Heldin, die sich in einer Welt bewegt und damit interagiert. Der oder die Heldin hat gewisse Eigenschaften, welche einen Einfluss auf den Spielverlauf haben. Das Spiel stellt jeweils eine Frage, auf die auf mindestens 3 verschiedene Arten reagiert werden kann.

Beispiel:

«Der Ritter fragt, ob Sie ihm helfen können. Wollen Sie ihm helfen?»

Ja Sicher (2) Nein (3) Das Pferd vom Ritter stehlen

Eigenschaften:

Nachdem man dem Ritter das Pferd gestohlen hat, ist man schneller, aber verletzt.

Aufgabenstellung

- Erstellen Sie ein Textbasiertes Spiel mit mindestens 6 verschiedenen Aufgaben.
- Erstellen Sie einen eigenen Plot (muss keinen Sinn ergeben) und zeigen Sie die Idee der Lehrperson. Die Lehrperson gibt den Entwurf zur Umsetzung frei.
- Der Held oder die Heldin muss Eigenschaften besitzen und je nach Lösung einzelnen Aufgaben muss sich die Eigenschaft ändern.
- Implementieren Sie das Spiel.

Datenauswertung



Erstellen Sie eine Software, welche Daten aus einer Datenquelle liest
(Bsp: Web API, Webseite, RSS-Feed, Dokument...) , diese Daten nach mindestens einem
Filterkriterium filtert und dann in eine Datei in einem auswählbaren Format (JSON, Excel, XML,
HTML...) schreibt und in der Konsole darstellt.



- Die einzelnen Komponenten sind austauschbar implementiert.
- Der Filterwert muss in der Konsole eingegeben werden können.
- Es müssen mindestens zwei Formate zur Auswahl stehen.

Beispiele:

Stundenplan:

- Lesen der Stundenplandaten aus der BBB-Webseite.
- Eingabe der Klasse

Filme

Lesen von Film-Daten aus einer Movie API
Filtern der Daten auf ein Erscheinungsjahr

Auswerten von Aktienkursen

Fragestellung: Ist die Änderung von Aktienkursen Saisonbedingt? Bsp. Sinken die Aktien immer im Januar)

- Lesen von Aktienkursen von API
- Filtern der Daten auf eine konfigurierbare Veränderung.
- Auswertung des Vorkommnisses pro Monat.

Sprechen Sie die Idee mit ihrer Lehrperson ab. Die Umsetzung kann erst beginnen, wenn die Lehrperson das OK gegeben hat.